Modeling Human Activity States Using Hidden Markov Models

Formative 2 – Hidden Markov Models · Machine Learning

Repository: https://github.com/munhugues/HMM_Formative2

Video: link

1. Background and Motivations

Human activity recognition (HAR) turns raw smartphone motion signals into meaningful activity labels. It underpins fall and inactivity alerts for older adults living alone, exercise and step tracking, and context-aware phones that adapt to what the user is doing. We study the core of that problem: telling apart four states – **standing, still (phone lying on a flat surface), walking and jumping** – from the accelerometer and gyroscope of one phone. The true activity is hidden behind noisy, high-rate measurements, and activities persist over time instead of switching at random. This is exactly the structure of a Hidden Markov Model (HMM): hidden states with sticky transitions that emit observable feature patterns. We implement the HMM from scratch in NumPy to understand how inference and learning work, and measure how well it recognises recordings it has never seen.

2. Data Collection and Pre-processing

Collection. We recorded **200 sessions** with Sensor Logger (accelerometer, gyroscope, total acceleration): **50 per activity** – standing with the phone at waist level, walking at a steady pace, continuous jumping, and the phone lying flat on a table. Raw recordings last 4.05–9.24 s (17 of the 200 are slightly shorter than the 5 s target). Raw folders are in data/raw/, the cleaned and labelled CSVs with a timestamp column in data/processed/.

Phone / OS	App	Requested rate	Measured rate (mean, range)
SM-G973F (Samsung, Android)	Sensor Logger 1.68	100 Hz (10 ms)	99.84 Hz (98.17–100.10)

Video evidence (5 s each, full body and face visible, phone held and recording): Standing · Walking · Jumping · Still [link]

Activity	Files	Raw signal (s)	Clean signal (s)	Windows (1 s)
Standing	50	287	233	388
Still (standstill)	50	316	261	450
Walking	50	287	234	395
Jumping	50	304	249	422

Table 2. Dataset after cleaning. Every activity has about 4 minutes of clean signal (requirement: 1 min 30 s).

Pre-processing. (i) **Sampling-rate harmonisation:** accelerometer and gyroscope are logged as separate streams with their own timestamps, with occasional gaps (0.04% of gaps exceed 15 ms, largest 100 ms). All streams are linearly interpolated onto one regular **100 Hz grid** – equal to the measured rate of our phone, so almost no information is invented; other phones go through the same function. (ii) **Trimming:** 0.5 s is removed at both ends of every recording to drop the start/stop taps and a one-sample gyroscope spike that appears at the start of some files. (iii) **Windowing:** 1.0 s windows (100 samples) with 50% overlap (hop 0.5 s). The size follows from the sampling rate and the activities: walking (≈2 steps/s) and jumping (≈2–4 jumps/s) always contain at least two movement cycles per window, a 1 s window gives 1 Hz FFT resolution (Nyquist 50 Hz, far above body motion), and longer windows would leave very few windows in 5 s recordings and blur activity boundaries.

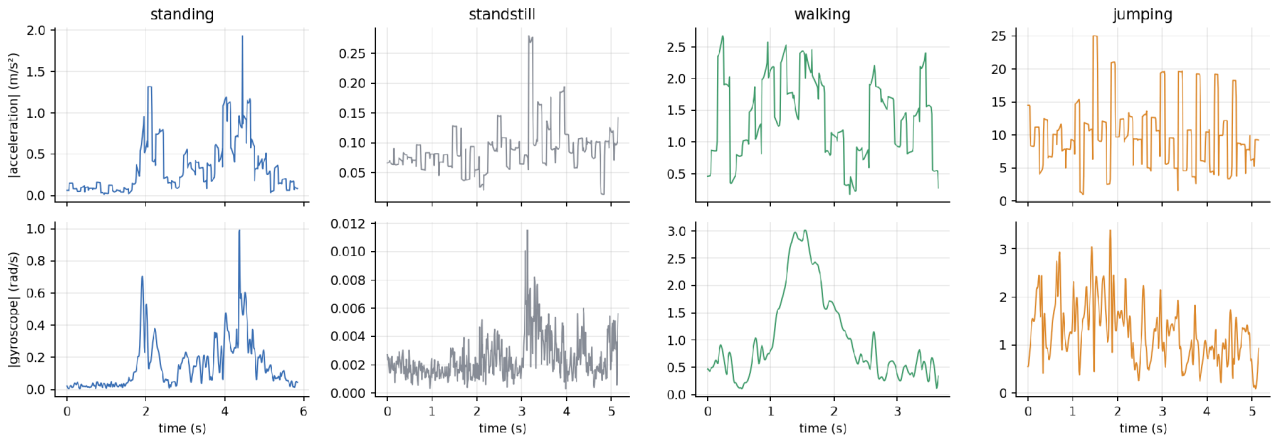


Figure 1. One recording per activity: acceleration magnitude (top) and gyroscope magnitude (bottom) after cleaning. The lying phone is almost silent; jumping is roughly seven to eight times stronger than walking.

3. HMM Setup and Implementation

Features. Each window yields **11 features (8 time-domain, 3 frequency-domain)**, computed on acceleration and gyroscope magnitudes (insensitive to how the phone is held) and per axis. Table 3 justifies each one.

Feature	Domain	Why it separates the activities
Mean, std, RMS of |acc|	Time	Overall movement intensity: jumping ≫ walking ≫ standing > still
Signal magnitude area (acc, gyro)	Time	Intensity summed over all three axes; the standard HAR energy measure
Std, RMS of |gyro|	Time	Phone rotation: sway when walking, tumbling when jumping, ≈ 0 when lying flat
Mean correlation between x, y, z	Time	Vertical bouncing couples the axes; walking couples them less
Dominant frequency (FFT)	Frequency	Rhythm of the movement: ≈ 2 Hz walking vs ≈ 4 Hz jumping
Spectral energy (FFT, no DC)	Frequency	Total oscillation power; separates energetic from quiet states
Spectral entropy (FFT)	Frequency	Periodic walking concentrates power (low entropy); postural sway and noise spread it

Table 3. Features per window (a Hann window is applied before the FFT to limit leakage).

Normalisation. The eight intensity features span 3–5 orders of magnitude and are right-skewed, so they are **log10-transformed**, then all 11 features are **Z-scored** with the mean and standard deviation of the training windows only (re-used unchanged on test data, so no test statistics leak). Z-scoring matches the Gaussian emission model and prevents large-valued features from dominating the likelihood.

Model components. Hidden states **Z** = {standing, still, walking, jumping}; observations **X** = one 11-dimensional normalised feature vector per window; transition matrix **A** (4×4, A[i,j] = P(Z(t+1)=j | Z(t)=i)); emissions **B** = one diagonal Gaussian $N(\mu_k, \sigma^2_k)$ per state; initial distribution **π**. Inference runs in log space (log-sum-exp) and was verified against brute-force enumeration of all state paths on a small random model.

Algorithms and how they tie to the metrics. Viterbi returns the single most probable state sequence, so it produces the per-window labels from which sensitivity, specificity, accuracy and the confusion matrix are computed. **Baum–Welch** (forward–backward EM) learns A, B and π by maximising the log-likelihood, which is also its convergence diagnostic: iteration stops when the gain falls below ε = 0.01. We initialise from labelled windows so every hidden state is anchored to a known activity (**primary model: supervised initialisation + Baum–Welch**) and compare against the label-only fit and a random-initialisation EM.

Train/test protocol. Each recording contains one activity, so transitions only exist if recordings are joined. We hold out **10 whole recordings per activity** (seeded) as unseen data and stitch them into **two test files** (166 and 159 windows, 98 s and 95 s, five recordings of each activity in random order). The other 40 recordings per activity are stitched into 8 training sessions (1330 windows). No window of a test recording is ever used for training or for fitting the scaler.

4. Results and Interpretation

Unseen data and generalisation. The test files contain recordings that were not used in training (new recordings from the same phone and person, not a different participant or environment). Pooled over both files (325 windows) the primary model reaches **94.15% overall accuracy** (92.2% and 96.2% per file); the label-only fit reaches 96.0%. The model therefore generalises well to new recordings, but this does not show generalisation to other people, phone positions or phones.

State (Activity)	Number of Samples	Sensitivity	Specificity	Overall Accuracy*
Standing	81	85.2%	97.1%	94.2%
Still (standstill)	86	91.9%	100.0%	97.8%
Walking	70	100.0%	95.3%	96.3%
Jumping	88	100.0%	100.0%	100.0%
All states	**325**	**94.2% (weighted)**	**–**	**94.15%**

*Table 4. Per-state results on the two unseen test files (primary model). Samples = 1 s windows. *Per-state values are one-vs-rest accuracies; the last row is multi-class accuracy.*

Variant	5-fold CV accuracy (training recordings) %	Test accuracy (unseen files) %	Notes
No transitions (per-window Gaussian)	97.4 ± 0.8	96.0%	ignores A
Label-only (supervised, no EM)	97.1 ± 1.0	96.0%	counts + sample moments
Supervised init + Baum–Welch (primary)	93.5 ± 1.5	94.2%	14 EM iterations
Random init + Baum–Welch	93.4 ± 1.5	94.2%	same optimum as primary

Table 5. Model variants. Cross-validation uses only training recordings (folds split by recording); the test files were evaluated once.

Table 4 shows **jumping** is decoded perfectly and **walking** has 100% sensitivity; all 19 errors are standing windows decoded as walking (12) or still windows decoded as standing (7), and they sit in only 4 of the 40 test recordings (Figure 5). Baum–Welch converged smoothly (log-likelihood -2280.3 → -1441.1, Figure 3) and a random initialisation reaches the identical optimum, yet EM lowers accuracy relative to the label-only fit: it maximises likelihood, not accuracy, and tightens the standing variances so that restless standing windows drift towards the walking Gaussian.

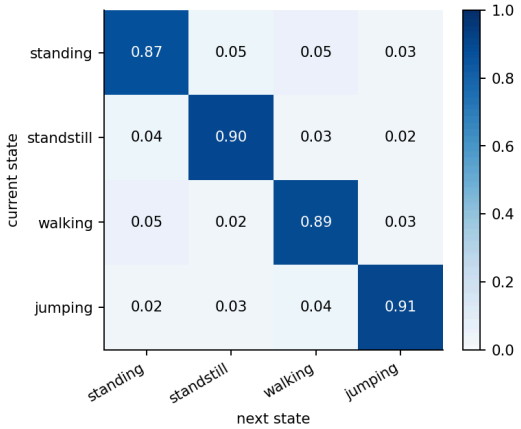


Figure 2. Learned transition matrix A (heatmap).

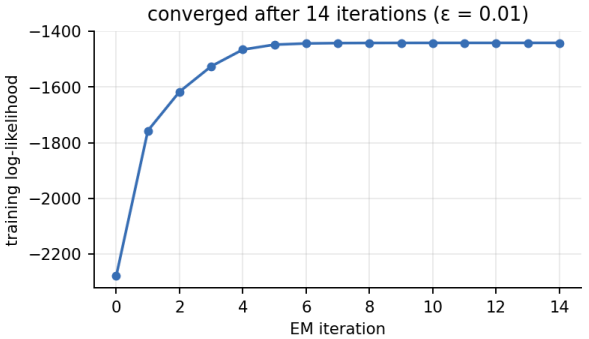


Figure 3. Baum–Welch log-likelihood per iteration (ε = 0.01).

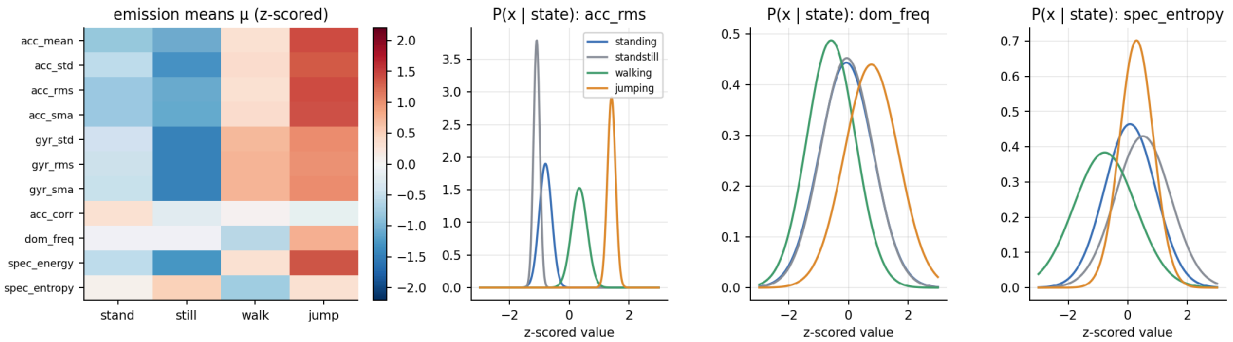


Figure 4. Emission model B: mean of every feature per state (left) and per-state Gaussians for three features (right).

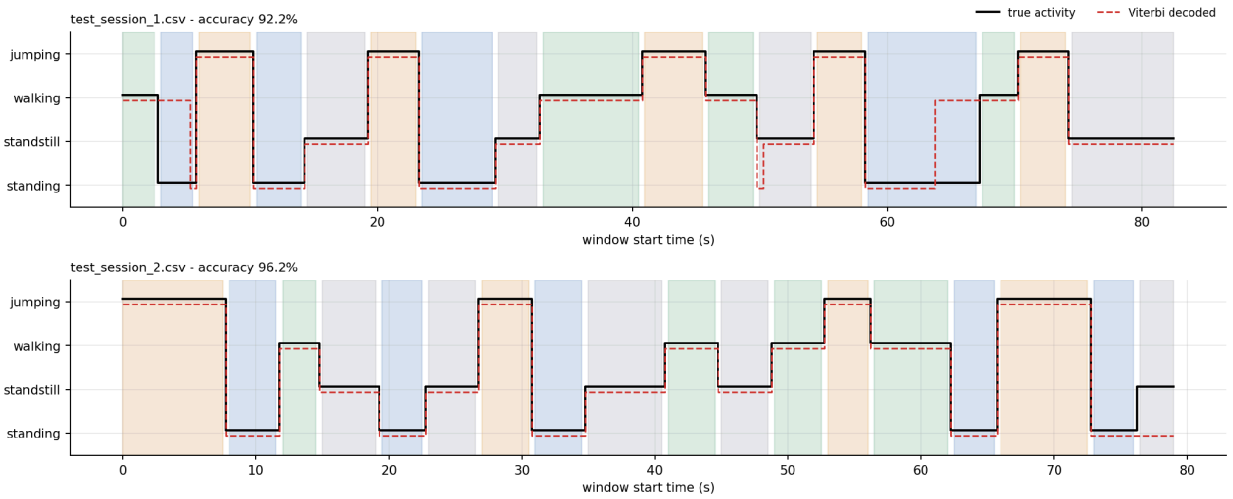


Figure 5. Viterbi-decoded activity sequence (dashed red) against ground truth (black) for the two unseen test files; background colours mark the true activity.

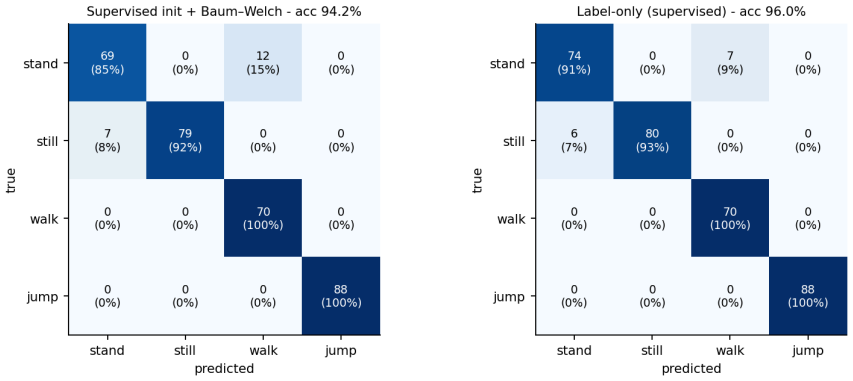


Figure 6. Confusion matrices on the unseen test files (counts and row percentages).

5. Discussion and Conclusion

Easiest and hardest activities. Jumping and walking are easy: they dominate acceleration energy, gyroscope energy and dominant frequency. Standing (sensitivity 85.2%) and still (91.9%) are hard because both are low-energy and their frequency features are essentially noise. The errors are not random but come from atypical recordings: two restless-standing recordings (mean acceleration RMS 0.45 and 0.50 vs a class mean of 0.20) were decoded as walking, and one recording of the phone lying flat on a vibrating surface (RMS 0.17 vs 0.07, but tilt 0.85° and gyroscope ≈ 0) was decoded as standing in all 6 windows.

Transition probabilities. The learned A has a strong diagonal (0.87, 0.90, 0.89, 0.91) and near-uniform small off-diagonal entries (0.02–0.05), i.e. an average stay of 3.8–5.4 s per state. This is realistic in one sense – activities persist – but it mostly reflects our protocol: each recording lasts about 4.9 s (≈ 9 windows, so the diagonal ≈ 1 − 1/9) and the stitching order was random. A realistic A (standing → walking likelier than standing → jumping, long dwell times) needs continuous recordings with natural transitions. Consistently,

ignoring A scores 97.4 ± 0.8% in CV, within one standard deviation of the HMM (97.1 ± 1.0%): the temporal model adds little because the windows are already well separated.

Sensor noise and sampling rate. The phone delivered 99.84 Hz on average and only 0.04% of gaps exceeded 15 ms, so interpolation was sufficient. Re-running the full pipeline after decimating to **50 Hz and 25 Hz** left test accuracy unchanged (label-only 96.0% / 96.0% / 96.2% at 100 / 50 / 25 Hz), so any phone sampling at ≥ 25 Hz should work after harmonisation. **Noise is far more damaging:** adding white noise with $\sigma = 0.05$ (m/s^2, rad/s) to the test signals drops label-only accuracy from 96.0% to 71.4% because every still window is decoded as standing – a phone lying flat is defined by an almost noise-free signal (gyroscope RMS ≈ 0.002 rad/s) and the log-scaled intensity features are very sensitive to the noise floor.

Improvements. (1) Record several people, phone placements and phones, and evaluate leave-one-person-out. (2) Use richer emissions (full covariance or a two-component mixture for standing, covering calm vs restless) and regularise EM with variance priors. (3) Record continuous sequences with real transitions, and add states such as sitting. (4) Add orientation information: a gravity-tilt feature from the total-acceleration stream raised label-only CV accuracy only from 97.1% to 97.7% (within one standard deviation) but cleanly separates a flat phone from a held one. (5) Train with noise augmentation.

Conclusion. A Gaussian HMM implemented from scratch recognises four activities in unseen recordings with about 94–96% accuracy, with perfect jumping and walking recall. The main lessons are that likelihood-optimal parameters are not accuracy-optimal, that the hard cases are atypical low-motion recordings, and that the noise floor of the sensor matters more than the sampling rate. All figures and numbers are reproducible with the notebook in the repository.